Genetic Algorithm

Travelling Salesman Problem:

```
import random
import math


def distance(a, b):
    return math.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)



def total_distance(chromosome, cities):
    total = 0
    n = len(chromosome)

    for i in range(n):
        city1 = cities[chromosome[i]]
        city2 = cities[chromosome[(i + 1) % n]]
        total += distance(city1, city2)

    return total



def fitness(chromosome, cities):
    return 1 / total_distance(chromosome, cities)



def selection(population, cities):
    weights = [fitness(ch, cities) for ch in population]
    return random.choices(population, weights=weights, k=len(population))


def pmx(parent1, parent2):
    n = len(parent1)
```

```
 a, b = sorted(random.sample(range(n), 2))

 def create_child(p1, p2):
     child = [None] * n


     child[a:b+1] = p1[a:b+1]


     for i in range(a, b + 1):
         if p2[i] not in child:
             pos = i

             while True:
                 value = p1[pos]
                 pos = p2.index(value)

                 if child[pos] is None:
                     child[pos] = p2[i]
                     break


     for i in range(n):
         if child[i] is None:
             child[i] = p2[i]

     return child

 child1 = create_child(parent1, parent2)
 child2 = create_child(parent2, parent1)

 return child1, child2
```

```
def rotate_mutation(chromosome):
    n = len(chromosome)

    if n < 2:
        return chromosome

    a, b = sorted(random.sample(range(n), 2))


    chromosome[a:b+1] = (
        chromosome[b:b+1] + chromosome[a:b]
    )

    return chromosome


def genetic_algorithm(cities, pop_size=100, generations=500,
                      crossover_prob=0.8, mutation_prob=0.08):

    n = len(cities)


    population = []

    for i in range(pop_size):
        chromosome = list(range(n))
        random.shuffle(chromosome)
        population.append(chromosome)


    best = min(population,
               key=lambda ch: total_distance(ch, cities))

    for gen in range(generations):
```

```
    parents = selection(population, cities)

    offspring = []


    for i in range(0, pop_size, 2):

        p1 = parents[i]
        p2 = parents[(i + 1) % pop_size]

        if random.random() < crossover_prob:
            c1, c2 = pmx(p1, p2)
        else:
            c1, c2 = p1[:], p2[:]


        if random.random() < mutation_prob:
            rotate_mutation(c1)

        if random.random() < mutation_prob:
            rotate_mutation(c2)

        offspring.extend([c1, c2])


    new_population = []

    for i in range(pop_size):
        parent = parents[i]
        child = offspring[i]

        if total_distance(parent, cities) < total_distance(child, cities):
            new_population.append(parent)
        else:
            new_population.append(child)
```

```
            if total_distance(current_best, cities) < total_distance(best, cities):
                best = current_best[:]

            worst_index = max(
                range(pop_size),
                key=lambda i: total_distance(population[i], cities)
            )

            population[worst_index] = best[:]

        return best, total_distance(best, cities)


n = int(input("Enter number of cities: "))

cities = []

print("Enter x and y coordinates of each city:")

for i in range(n):
    x, y = map(float, input().split())
    cities.append((x, y))

best_route, minimum_distance = genetic_algorithm(cities)

print("\nBest Route:")

for city in best_route:
    print(city, end=" -> ")

print(best_route[0])

print("Minimum Distance:", round(minimum_distance, 2))
```

Output:

```
Enter number of cities: 5
Enter x and y coordinates of each city:
0 0
2 4
5  2
6  6
8 3

Best Route:
2 -> 0 -> 1 -> 3 -> 4 -> 2
Minimum Distance: 21.1
```